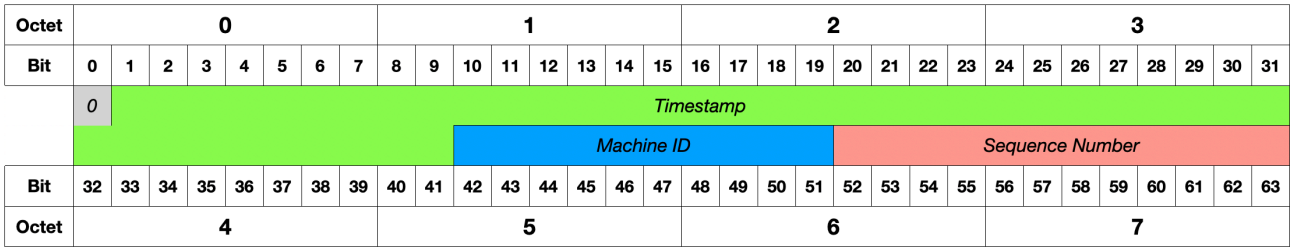
Unique ID Generators

systemdesignschool.io/fundamentals/unique-id-generators



A single database auto-increment sequence works fine on one server. But when Server 1 and Server 2 both generate ID 1 for different users, the system breaks. Every tweet, video, and shortened URL needs a globally unique identifier that can be generated independently on any server without collisions.

Why Auto-Increment Fails

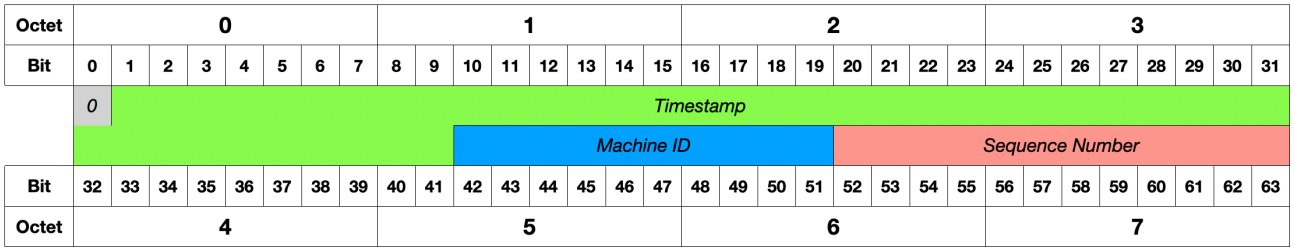
Auto-increment sequences require coordination. Each server must ask a central authority for the next ID. This creates a bottleneck. The central ID generator becomes a single point of failure. Every ID request requires a network call, adding latency to every write operation.

Distributed ID generation needs different properties. IDs must be unique across the entire system. Generation should not fail when individual servers go down. ID creation should be fast, ideally under one millisecond. The system should work with thousands of servers generating millions of IDs. For many applications, newer IDs should be greater than older ones to enable time-based sorting.

Snowflake IDs

Snowflake solves the distributed ID problem by embedding uniqueness into the ID structure itself. Instead of coordinating between servers, each server generates IDs independently using a clever bit allocation strategy. If every server has a unique identifier and includes a timestamp, IDs will be unique across the entire system without coordination.

A Snowflake ID uses 64 bits divided into four parts: 1 sign bit, 41 bits for timestamp, 10 bits for machine ID, and 12 bits for sequence number. The timestamp stores milliseconds since a custom epoch, providing roughly 69 years of timestamps. Different times produce different IDs. This makes IDs sortable by creation time.



The machine ID is a unique identifier per server. It supports up to 1,024 different servers. Each server gets a unique ID during deployment. Different machines produce different IDs even at the same millisecond. This requires coordination at setup time to ensure no two servers share the same machine ID.

The sequence number is a counter within the same millisecond. It handles multiple ID requests in the same millisecond on the same machine. Each machine can generate up to 4,096 IDs per millisecond. The counter resets to zero each new millisecond.

A concrete example shows how this works. At time 1640995200000, Server 1 with machine ID 001 generates ID ending in 0001. The next request on the same server in the same millisecond gets 0002 by incrementing the sequence. Server 2 with machine ID 002 generates an ID ending in 0001 at the same millisecond because it has a different machine ID. When the next millisecond arrives, the sequence resets to 0001.

Snowflake Implementation Example

Here's an interactive implementation showing the key concepts:

Snowflake ID Generator

Interactive implementation showing distributed unique ID generation with Twitter-like example

Loading Python environment... This may take a few seconds on first load.

Snowflake Trade-offs

Snowflake excels at independent ID generation. Each server generates IDs without coordination, achieving 2-3 million IDs per second per machine. The IDs are time-sortable, meaning newer tweets have larger IDs. They are roughly sequential, which benefits database B-tree performance during inserts.

The approach has limitations. It depends on synchronized clocks across servers. The system supports only 1,024 machines and 4,096 IDs per millisecond per machine. Clock skew creates problems. If a server clock goes backwards, it can generate duplicate IDs. Machine ID

coordination is required at setup to ensure each server has a unique machine ID.

> **Machine ID Coordination Explained:**
>
> **The Problem**: Every server needs a unique machine ID to prevent collisions.
>
> **Bad scenario (no coordination)**:
>
> ```
> Server A starts up → picks machine ID 5
> Server B starts up → picks machine ID 5  ← Problem! Same ID!
> Both generate IDs at same time → Collision guaranteed
> ```
>
> **Common coordination strategies**:
>
> 1. **Configuration Management**:
>
>    ```
>    # server-config.yml server_1: machine_id: 1 server_2: machine_id: 2
>    server_3: machine_id: 3
>    ```
> 2. **Service Discovery + Database**:
>
>    ```
>    # Server startup process def get_machine_id(): # Try to get existing
>    ID from database existing_id = db.get("machine_id", server_hostname)
>    if existing_id: return existing_id # Allocate new ID next_id =
>    db.increment("next_available_machine_id") db.set("machine_id",
>    server_hostname, next_id) return next_id
>    ```
> 3. **Container Orchestration**:
>
>    ```
>    # Kubernetes assigns unique IDs via environment variables docker run -
>    e MACHINE_ID=${POD_ID} snowflake-service
>    ```
>
> **Why this is "coordination"**: Unlike UUID (completely independent), Snowflake requires a setup step where servers agree on who gets which machine ID.

Use Snowflake when you need sortable IDs like Twitter timelines. It works well for high-throughput systems generating millions of IDs per second. The pattern suits distributed systems with a known number of servers under 1,024. It fits applications where the ID structure can be exposed, though this reveals the timestamp.

Alternative Unique ID Strategies

Snowflake isn't the only solution for distributed ID generation. Different approaches have different trade-offs that make them better suited for specific use cases.

UUID

UUID generates 128-bit random or pseudo-random numbers with extremely low collision probability. An example UUID v4 looks like `f47ac10b-58cc-4372-a567-0e02b2c3d479`. UUIDs are truly distributed with no coordination needed. They have no clock dependency and are standardized across programming languages. You can generate them offline.

The downsides are significant for databases. UUIDs are 16 bytes compared to 8 bytes for Snowflake. They are not sortable by time. They cause poor database performance due to random inserts. They are not human-readable.

Use UUIDs for distributed systems where you cannot coordinate machine IDs, offline applications, or systems that do not need time-based sorting.

UUIDs hurt write performance because they are completely random. Sequential inserts from Snowflake or auto-increment always append to the end of the B-tree. Random UUID inserts scatter across the tree, requiring the database to find the correct position with more disk reads, split pages when inserting in the middle, and create index fragmentation that reduces cache efficiency. Sequential inserts can be 5-10x faster than random UUID inserts in write-heavy systems.

For a deeper understanding of why sequential writes perform better than random writes in databases, see Data Structures Behind Databases.

ULID

ULID generates 128-bit IDs with 48-bit timestamp plus 80-bit randomness, encoded in Base32. An example ULID looks like `01F8VYXK67BGC1XPD2YH1W8HTH`. ULIDs are time-sortable like Snowflake. They use case-insensitive, URL-safe encoding. No machine ID coordination is needed. The 48-bit timestamp provides 8,925 years of range.

ULIDs are larger than Snowflake, using 26 characters versus 19 digits. They support less throughput per millisecond than Snowflake. As a newer standard, they have less tooling support.

Use ULIDs when you need time-sortable IDs but cannot manage machine ID coordination, or when you want human-readable IDs.

Partitioned Auto-Increment

Partitioned auto-increment uses database sequences with different starting points and increments. Server 1 generates 1, 4, 7, 10, 13 with start=1 and increment=3. Server 2 generates 2, 5, 8, 11, 14 with start=2 and increment=3. Server 3 generates 3, 6, 9, 12, 15 with start=3 and increment=3.

This approach is simple to implement with guaranteed sequential ordering. IDs are small at 8 bytes with native database support. The database becomes a bottleneck for ID generation. Adding or removing servers is difficult. The ID sequence reveals information about system scale. The database is a single point of failure.

Use partitioned auto-increment for smaller systems when you need guaranteed sequential IDs and prefer simple architectures.

Choosing the Right ID Strategy

Requirement	Snowflake	UUID v4	ULID	Auto-Increment
Time-sortable	✅	❌	✅	✅
No coordination	❌*	✅	✅	❌
High throughput	✅	✅	✅	❌
Small size	✅	❌	❌	✅
Human-readable	❌	❌	✅	✅
No clock dependency	❌	✅	❌	✅

*Snowflake needs machine ID coordination at setup, but no runtime coordination between servers

System Design Interview Decision Framework

Ask these questions to choose the right approach:

1. **Do you need time-based sorting?**
    - Yes → Snowflake, ULID, or Auto-increment
    - No → UUID v4
2. **How many servers will you have?**
    - < 1,000 servers → Snowflake works well
    - > 1,000 servers → Consider ULID or UUID
3. **Can you coordinate machine IDs?**
    - Yes → Snowflake is a good choice
    - No → ULID or UUID
4. **Do you need IDs to be unguessable?**
    - Yes → UUID (fully random)
    - No → Snowflake or ULID
5. **Is this a read-heavy or write-heavy system?**
    - Write-heavy → Avoid UUID (random inserts cause B-tree page splits, 5-10x slower)
    - Read-heavy → UUID is fine (insert performance doesn't matter much)

Twitter uses Snowflake for tweets because it needs time-sorting at high scale. Instagram uses a modified Snowflake with different bit allocation. GitHub uses UUID for some resources and auto-increment for others. Stripe uses custom prefixed IDs like `cus_1234567890abcdef`, which are not UUIDs but their own format.

Understanding these trade-offs helps you make informed decisions in system design interviews and shows deeper architectural thinking.

When Standard Solutions Don't Fit: Custom ID Systems

Sometimes none of the standard approaches (Snowflake, UUID, ULID, Auto-increment) perfectly match your requirements. Many successful companies create custom ID formats tailored to their specific needs.

Stripe's Approach

Stripe uses prefixed random IDs with the format `{prefix}_{random_string}`. Examples include `cus_1234567890abcdef` for customers, `ch_3L4E5Y2eZvKYlo2C` for charges, `sub_1A2B3C4D5E6F7G8H` for subscriptions, and `inv_1GjJKl2eZvKYlo2C` for invoices.

This format is human-readable. You immediately know what type of object it is. The IDs are URL-safe, using only characters that do not need encoding in URLs.

> **What "URL-safe" means:**
>
> ✅ **Safe characters**: a-z, A-Z, 0-9, hyphen (-), underscore (_)
> ❌ **Unsafe characters**: +, /, =, spaces, &, ?, #, %
>
> **Examples:**
>
> ```
> UUID:     f47ac10b-58cc-4372-a567-0e02b2c3d479  ✅ (hyphens are safe)
> Base64:   SGVsbG8gV29ybGQ+              ❌ (+ and / need URL encoding)
> Stripe:   cus_1234567890abcdef           ✅ (only letters, numbers, underscore)
> ```
>
> **Why this matters:**
>
> - Can use directly in URLs: `api.stripe.com/customers/cus_123abc`
> - No encoding needed: `fetch('/api/orders/ord_456def')`
> - Avoids bugs from forgotten URL encoding

GitHub's Approach

GitHub uses different ID strategies for different use cases. Repository URLs use human-readable names like `github.com/user/repo-name`. Issue IDs use auto-increment per repository, showing as issue #1, #2, #3. Commit SHAs use Git hashes as required by distributed version control.

YouTube's Approach

YouTube uses short random IDs like `dQw4w9WgXcQ` with 11 characters in Base64-like encoding. This creates short URLs like `youtube.com/watch?v=dQw4w9WgXcQ`. The IDs are non-sequential, preventing users from guessing other videos by incrementing. Eleven characters provide roughly 64 bits of randomness. The IDs are memorable enough to share in text messages.

When to Build Custom IDs

Consider custom IDs when none of the standard approaches fit. User-facing requirements might need human-readable or branded formats. Systems with multiple object types might want to distinguish customers from orders from products. API design might require stable external IDs separate from internal database IDs. Regulatory requirements might demand audit trails or specific ID formats. Migration needs might require changing internal storage without breaking external APIs.

Custom ID Generator

Build your own ID system like Stripe - experiment with different prefixes, lengths, and character sets

```
def __init__(self, prefix, length=16, include_uppercase=False):
self.prefix = prefix
self.length = length
# Build alphabet based on options
self.alphabet = "0123456789abcdefghijklmnopqrstuvwxyz"
if include_uppercase:
self.alphabet += "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
def generate(self, count=1):
"""Generate one or more IDs"""
for _ in range(count):
random_part = ''.join(secrets.choice(self.alphabet)
for _ in range(self.length))
ids.append(f"{self.prefix}_{random_part}")
return ids if count > 1 else ids[0]
def generate_batch(self, count):
"""Generate a batch of IDs efficiently"""
return [self.generate() for _ in range(count)]
def demo_stripe_like_system():
"""Demonstrate a Stripe-like ID system"""
print("=== Stripe-like Custom ID System ===")
# Create generators for different resource types
customer_gen = IDGenerator("cus", length=16)
charge_gen = IDGenerator("ch", length=18)
subscription_gen = IDGenerator("sub", length=16)
invoice_gen = IDGenerator("inv", length=16)
product_gen = IDGenerator("prod", length=12)
```

Loading Python environment... This may take a few seconds on first load.

Interview Application

When discussing custom ID systems in interviews, start with standard options. Mention evaluating Snowflake, UUID, and ULID first. Then explain why they do not fit the requirements. For example, users need to share URLs, so UUID is too long. Show trade-off thinking by noting that custom IDs mean more code to maintain, but better user experience makes it worthwhile. Discuss implementation details like using cryptographically secure randomness to prevent enumeration attacks. Mention maintenance considerations, noting that the team would need to ensure uniqueness and handle edge cases themselves.